# RISC-V 与 PE 工程入门笔记

零基础详细版：从一条 **C** 语句，走到电路算出答案。

本文对应 `/home/zhz/contest/task1/my_design`，使用工程中的实际代码。教学小例子会单独标明。阅读时可以一边看本文，一边打开对应源文件。

第一遍先读第1～7节，弄清“谁在工作、地址是什么、函数怎样调用”。第二遍再读电路部分。完整程序和比赛术语放在附录，方便读懂主线后复习。

## 1. 我们到底要让这套系统做什么？

先把比赛和复杂模型放在一边。我们当前研究的是一个很小的测试：把数字交给计算电路，再把答案读回来，看看算得对不对。

先看一个计算单元做的事情：

> 输入是2，权重是1。
> 做一次乘法：2 × 1 = 2。
> 再把这个2加到已有的结果上。

“权重”先理解为参加乘法的另一个数。例如输入2、权重3，就是算 `2×3`。

真实测试会让16行计算单元接力相加。每行加上一个2，最后得到32。与此同时，还计算另外三路，最后得到48、64、80。

你暂时只需要带着一个问题往下看：程序怎样把输入和权重送进电路，又怎样确认读回的**32**是正确的？

## 2. 先认识三个角色，以及它们在哪里运行

### 2.1 CPU、PE、AXI各自做什么

**CPU**负责按程序安排步骤。 程序让它写数据，它就执行写入；程序让它读结果，它就执行读取。

RISC-V是CPU识别指令时遵守的一套规则。本工程采用的具体CPU设计叫VeeR EL2。

**PE**负责计算。 PE是计算单元的简称。本工程一个PE中有权重存储器、四个乘法器和四个加法器。

**AXI**是交换数据时遵守的通信规则。 CPU的访问请求，需要携带地址、数据等信息，并得到响应。相关接口电路按照AXI规则完成这些传递。

可以把它们的配合看成：

```
CPU：按程序准备材料，并安排操作
                    ↓ 通过AXI接口
PE：收到数据，执行乘法和加法
                    ↓ 通过AXI接口
CPU：读取答案，与预期结果比较
```

“接口”可以先理解为模块与外界交换信息的入口和出口。

## 2.2 为什么工程里既有C，又有Verilog

你写一份操作步骤，和你设计一台机器，是两件相互配合的工作。

| 在做什么 | 使用的语言 | 工程中的例子 |
| --- | --- | --- |
| 告诉CPU依次执行哪些操作 | C | `pe_smoke.c` |
| 描述机器内部有哪些电路、怎样连接 | Verilog或SystemVerilog | `pe_1sram_4mul.v`、`soc_top.sv` |

C程序要经过编译，转换成CPU能识别的机器指令，然后由CPU执行。

Verilog/SystemVerilog描述的电路，可以交给仿真工具，在电脑中模拟它的工作。这样的电路代码通常称为RTL。

你当前打开的 `.v`、`.sv` 文件描述电路；`.c` 文件描述**CPU**要执行的步骤。

## 2.3 Ubuntu、仿真工具和电路里的CPU是什么关系

我们现在没有制造真实芯片，而是在电脑中模拟芯片：

```
Ubuntu/Linux环境
    ↓ 运行
VCS仿真工具
    ↓ 模拟
SoC电路，里面包含CPU和PE
    ↓ 其中的CPU执行
pe_smoke.c编译出来的机器指令
```

SoC可以理解为把CPU、计算模块和接口等连接起来的完整系统。`soc_top.sv` 主要描述这套系统的总连接关系。

# 3. 先把这一句读懂：wr(0x008, 0xffff)

## 3.1 两个参数分别回答两个问题

```
wr(0x008, 0xffff);
```

先按下面的意思读：

```
wr(去哪个控制位置, 往那里写入什么值);
```

因此，这句中：

- `0x008`：选择要访问的控制位置。
- `0xffff`：决定这个位置要保存的设置值。

可以类比 `设置(音量, 20)`：先选中“音量”这项设置，再把值改成20。

**`0x008`** 本身没有决定选哪一行；它只是指向**“行选择”**这项设置。具体选哪些行，由后面的值决定。

## 3.2 这个控制位置保存着16个开关状态

这个位置对应一组电路中的存储单元，保存16位行使能状态。你可以把它想成16个小开关，每个对应阵列的一行。

“使能”就是“允许它工作”。这里控制的是这一行各PE的权重存储器是否启用。

先只画其中4个开关，行号从0开始：

```
对应行号：第3行   第2行   第1行   第0行
开关状态：  0       0       0       1
```

这表示只选中第0行，也就是平常说的第一行。

看三个例子：

| 写入的值 | **16**个开关的状态 | 哪些行启用 |
| --- | --- | --- |
| `0x0001` | 最低一位是1，其余是0 | 第0行 |
| `0x0003` | 最低两位是1，其余是0 | 第0行和第1行 |
| `0xffff` | 16位全部是1 | 全部16行 |

所以，`wr(0x008, 0xffff)` 的意思是：把**“行选择”**设置中的**16**个开关全部设为**1**。 这些状态是保存设置的标记，实际计算单元在PE阵列中。

### 3.3 为什么实际地址不是0x008

程序中先定义了PE的起始地址：

```
#define PE_BASE 0x90000000u
```

把PE的接口想成一栋有很多窗口的楼。`PE_BASE` 指定这栋楼的起点，`0x008` 指定从起点往后找多远。这个距离叫“偏移”。

```
PE起始地址：0x90000000
加上偏移：    0x008
实际地址： 0x90000008
```

因此CPU实际访问的是 `0x90000008`。工程中的路由电路根据这类地址，把请求送往PE接口。

程序通过地址访问硬件控制或数据接口，这种方式叫MMIO。先记住“通过读写指定地址使用硬件”即可。

小练习： 把 `wr(0x008, 0xffff)` 改成 `wr(0x008, 0x0001)`，哪个部分变了？

答案：访问位置没变，写入的设置值变了，所以从“全部16行启用”变成“只有第0行启用”。

## 4. 看懂十六进制、8位、32位和地址加4

### 4.1 0x只是数字写法的提示

`0x` 表示后面的数字按十六进制解释。`0x008` 就是十进制的8；前面的零不会改变大小。

平常十进制只有0～9这十个数字。十六进制需要16种数字，所以数到9以后继续用字母：

| 十进制 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 |
|---|---|---|---|---|---|---|---|---|---|
| 十六进制 | 8 | 9 | a | b | c | d | e | f | 10 |

例如，`0x02c` 的数值是 `2×16+12=44`；`0x028` 是 `2×16+8=40`。两者相差4。

```
0x028 → 0x029 → 0x02a → 0x02b → 0x02c
       第1步     第2步     第3步     第4步
```

这里列出了5个地址，但从起点到终点只移动4步。

### 4.2 一位就是一个只能保存0或1的位置

可以想成一个小灯，只有灭和亮两种状态。用数字表示，就是0或1。

8个这样的位置，合起来叫一个字节：

```
1字节 = 8位

例如数字1，用8位保存：00000001
```

一个十六进制数字，正好可以表示4位二进制。因此上面的8位可以写成两个十六进制数字：

```
二进制分组：0000 | 0001
十六进制：   0 |    1
```

所以数字1的一个字节写法就是 `01`。

### 4.3 为什么0x01010101按两个数字一组

因为当前硬件规定：一个权重用8位保存。8位就是两个十六进制数字，所以按这个长度分组：

```
0x01010101
     ↓
01 | 01 | 01 | 01
     ↓
 1 |  1 |  1 |  1
```

例如，把它换成 `0x05040302`，就包含四个字节 `05、04、03、02`。这些字节从最低位到最高位的数值，依次是2、3、4、5。

数据在代码中是一起写入的，硬件根据“每份数据占8位”的约定分别使用。

### 4.4 为什么0xffff又表示16个开关

因为这里每个开关只需要1位。一个十六进制数字 `f` 对应二进制的四个1：

```
十六进制：  f    f    f    f
二进制：  1111 1111 1111 1111
```

因此是16个开关全部选中。同样是一个数字，如何分组、怎样使用，取决于所访问接口的规定。

再看选中两行的例子：0x0003写成16位二进制是 `0000 0000 0000 0011`。最右边两位是1，它们分别控制第0行和第1行。

### 4.5 为什么连续写入位置每次加4

当前C程序一次写入32位，也就是4字节。地址按字节编号，因此这次写入占连续4个字节位置。

```
第一次从0x020开始：
地址：0x020  0x021  0x022  0x023
数据： 01     01     01     01

下一组从0x024开始：
地址：0x024  0x025  0x026  0x027
数据： 01     01     01     01
```

四次写入的起始偏移就是 `0x020、0x024、0x028、0x02c`。

底层AXI接口的数据宽度是64位，会选择相应字节来完成这里的32位访问。先把“C程序每次写4字节”学清楚即可。

## 5. wr和rd怎样调用？为什么能反复用同一个名字？

### 5.1 函数就像一份可以重复使用的操作说明

定义函数，是写好“收到参数后怎么做”。调用函数，是现在交给它参数，请它执行一次。

`wr` 的实际定义在 pe_smoke.c：

```
static inline void wr(unsigned off, uint32_t v) {
    *(volatile uint32_t *)(PE_BASE + off) = v;
    __asm__ volatile ("fence iorw, iorw" ::: "memory");
}
```

先看懂名字和两个参数就够了：

| 名字 | 表示什么 | 一次调用中的例子 |
| --- | --- | --- |
| `wr` | 这份写入操作说明的名字 | 调用它执行一次写入 |
| `off` | 地址偏移 | `0x008` |
| `v` | 要写进去的值 | `0xffff` |

函数内部的主要动作是“向 `PE_BASE+off` 写入32位的v”。其中复杂的指针和汇编写法，放到附录再看。

读C代码时，先认识三个常见标记：分号表示这条语句结束；大括号把同一组操作包在一起；双斜线后面是给人看的注释。函数写成void返回类型，表示它不向调用处交回一个结果值；uint32_t表示32位无符号整数。

### 5.2 一次调用，究竟按什么顺序发生

假设主程序执行到：

```
wr(0x008, 0xffff);
```

从源码操作顺序理解，是下面五步：

> ① 主程序提供两个值：0x008和0xffff。
> ② 函数这次使用：off=0x008，v=0xffff。
> ③ 算出地址：PE_BASE+off=0x90000008。
> ④ CPU执行对应写入，并执行相关的顺序约束。
> ⑤ 这次操作结束，继续主程序中的下一句。

启动时，CPU先执行 `start.S` 里的准备代码，然后调用 `main()`。函数定义写在文件前面，只是预先写好操作说明，不表示它们会先依次运行。

### 5.3 同一个wr为什么有不同效果

因为 `wr` 一直做“地址写入”。硬件收到不同地址后，修改不同的设置。

| 调用 | **wr**所做的同一种动作 | 硬件识别地址后的作用 |
| --- | --- | --- |
| `wr(0x000, 0)` | 向对应地址写入数据 | 选择权重存储位置 |
| `wr(0x008, 0xffff)` | 向对应地址写入数据 | 更新行开关状态 |
| `wr(0x010, 0xffff)` | 向对应地址写入数据 | 发出权重保存操作 |

像快递员按地址送东西：投递动作相同，收件地址不同，就交给不同地方处理。这里没有使用函数重载。

### 5.4 rd参数是读取位置，返回值才是数据

`rd` 的定义是：

```
static inline uint32_t rd(unsigned off) {
    return *(volatile uint32_t *)(PE_BASE + off);
}
```

它从指定地址读取32位数据，并把数据返回给调用处。例如：

```
uint32_t result = rd(0x400);
```

按顺序理解：先调用rd读取结果，再把读到的数保存到变量result中。本测试结果稳定后，这里会读到32。

参数可以是常数，也可以是变量：

```
unsigned addr = 0x400;
uint32_t result = rd(addr);
```

上面和直接传 `0x400` 读的是同一处。还可以先计算表达式：

```
rd(0x400 + 16*col + 4*lane)
```

先用col和lane算出偏移，再交给rd。偏移是常数，并不意味着读回的数据是常数。

小练习： 在 `rd(0x400)` 中，`0x400` 是要读的答案吗？

答案：它指定从哪里读。真正的答案由硬件返回。

## 6. 按程序顺序，一步一步准备数据

这一节只看CPU安排什么操作。先不展开接口内部的复杂电路。

### 6.1 选择权重保存到哪个格子

```
wr(0x000, 0);
```

每个PE都有一块小的权重存储器，可以想成有16个带编号的格子，每格保存一个8位权重。这里选择的是每个PE存储器中的第0号格子。

执行后：后续权重写入使用存储位置0。

注意，这个“0号存储位置”和“阵列第0行”是不同的编号。一个是存储器里面的格子号，一个是PE在阵列中的行号。

### 6.2 启用哪些行的权重存储器

```
wr(0x008, 0xffff);
```

执行后：16个行使能状态全部变成1，即全部16行的权重存储器都启用。

这一步选好了哪些行参与后续权重操作。此时还没有把准备好的新权重保存进去。

### 6.3 准备16列的权重

原程序写的是：

```
for (unsigned i = 0; i < 4; ++i)
    wr(0x020 + 4*i, 0x01010101);
```

这个for循环让同一条操作重复执行。三个部分分别表示：

- `unsigned i = 0`：先让计数用的变量i等于0。
- `i < 4`：每次执行前，检查i是否小于4。
- `++i`：每执行完一次，让i加1，然后再次检查。

实际发生的四次调用：

| i的值 | 算出的偏移 | 这次准备哪几列的权重 |
| --- | --- | --- |
| 0 | `0x020+4×0=0x020` | 第0～3列 |
| 1 | `0x020+4×1=0x024` | 第4～7列 |
| 2 | `0x020+4×2=0x028` | 第8～11列 |
| 3 | `0x020+4×3=0x02c` | 第12～15列 |

i变成4时，`i<4` 不成立，循环结束。因此展开后就是：

```
wr(0x020, 0x01010101);
wr(0x024, 0x01010101);
wr(0x028, 0x01010101);
wr(0x02c, 0x01010101);
```

执行后：接口电路中已经准备好16个权重，全部为1。每个权重将送给对应的一列。

### 6.4 发出一次保存权重的操作

```
wr(0x010, 0xffff);
```

可以把前一步想成“把表格填好”，这一步是“按下保存”。

执行这次访问后，接口给全部16行发出写入控制。符合条件的时钟到来时，各PE把自己收到的列权重保存到第0号存储位置。

当前全部列权重都是1，所以全部选中PE的第0号格子里都保存1。

### 6.5 把每一路累加的起点设为0

```
for (unsigned i = 0; i < 64; ++i)
    wr(0x200 + 4*i, 0);
```

计算除了乘法，还要逐步相加。例如：

```
起点0 + 第一次的2 + 第二次的2 = 4
起点10 + 第一次的2 + 第二次的2 = 14
```

因此必须先确定从几开始加。本测试设成0。

16列中，每列有四路独立的累加过程，所以要设置 `16×4=64` 个起始值。一次写一个32位起始值，偏移每次加4。

### 6.6 给每一行送入四路输入

```
for (unsigned row = 0; row < 16; ++row)
    wr(0x100 + 4*row, 0x05040302);
```

row是行号，从0到15。每次写入给一行提供四个输入：

```
十六进制从左到右：05 | 04 | 03 | 02
硬件从最低8位开始取，也就是从最右边开始。

第0路=2，第1路=3，第2路=4，第3路=5。
```

前两次写入为：

```
wr(0x100, 0x05040302); // 给第0行四路输入：2、3、4、5
wr(0x104, 0x05040302); // 给第1行四路输入：2、3、4、5
```

执行完循环：全部16行都收到同样的四路输入。

这里的“四个输入”和前面的“四个权重”分配方式不同：一次权重写入准备四列的权重；一次输入写入准备一行的四路数据。

### 6.7 等计算结果稳定

```
settle();
```

这是程序自己定义的等待函数，让CPU执行一小段空转操作。CPU等待期间，PE仍随时钟工作，让数据逐步经过阵列。

PE中的运算电路已经存在并工作，settle只是留出时间。函数里循环128次，不代表精确等待128个时钟周期，因为循环本身也需要指令。

# 7. 看硬件前，先理解“保存数据”和“时钟”

## 7.1 寄存器是什么

寄存器是电路里能保存一份数据的部件。可以先想成一个小盒子，里面保留最近一次按要求存入的数。

在本工程的时钟控制代码中，寄存器在规定的时钟时刻接收新值。这个特点使数据能够分步传递。

## 7.2 时钟上升沿是什么

时钟信号clk反复在0和1之间变化：

```
时钟状态：0 → 1 → 0 → 1 → 0 → 1
                ↑       ↑       ↑
          这些从0变1的时刻，叫上升沿。
```

把它想成节拍器，每到一个规定节拍，寄存器保存当时的数据。

pe_1sram_4mul.v，第49行起中有实际代码：

```
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        data_reg   <= 32'b0;
        weight_reg <= 8'b0;
    end
    else begin
        data_reg   <= data_in;
        weight_reg <= weight;
    end
end
```

先分两种情况读：

- 正常运行：每次时钟上升沿，把data_in保存到data_reg，把weight保存到weight_reg。
- 复位时：rst_n为0，这两个寄存器清零。复位从1变0也会触发这个代码块，因此称为低有效异步复位。

这里清零的是这两个寄存器。SRAM里的测试权重仍要由软件按前面的步骤写入。

## 7.3 为什么数据经过多级寄存器需要多个节拍

`<=` 在这里表示安排寄存器更新。同一个时钟沿，大家先读取更新前的值，再更新自己的保存内容。

下面是专门解释这个概念的教学例子，并非新增到工程的代码：

```
a <= input_data;
b <= a;
```

假设这两句在同一个时钟上升沿触发，原来a、b都是0，现在input_data稳定为2：

| 时刻 | **a**保存的值 | **b**保存的值 |
| --- | --- | --- |
| 第一次上升沿之前 | 0 | 0 |
| 第一次上升沿更新后 | 2 | 0 |
| 第二次上升沿更新后 | 2 | 2 |

第一次时，b拿到的是a更新前的0。第二次时，它才能拿到a已经保存的2。这就是理解“数据逐级传递”的基础。

### 7.4 多个硬件模块可以同时工作

C主程序按程序流程安排访问。Verilog描述的多个乘法器和加法器则是同时存在的电路，它们可以同时工作。

文件里先写u_mul0、后写u_mul1，是在描述两个乘法器实例，不表示先调用完第一个函数再调用第二个。

这和时钟并不矛盾：各模块可以并行计算，相关寄存器在时钟到来时保存它们的结果。

## 8. 硬件怎样知道0x008应该修改哪些开关

### 8.1 接口先接收地址和数据，再选择操作

再次跟踪：

```
wr(0x008, 0xffff);
```

CPU发出的实际地址是0x90000008，写入数据是0xffff。AXI接口分别传递地址和数据，PE接口把两者接收好，再处理这次写入。

接口电路已经按设计规定了地址的含义：

| PE内部偏移 | 对应哪项设置 |
| --- | --- |
| `0x000` | 权重要存到SRAM哪个位置 |
| `0x008` | 哪些行的SRAM启用 |
| `0x010` | 哪些行执行一次SRAM写入 |
| `0x020` 起 | 准备好的各列权重 |
| `0x100` 起 | 各行的输入数据 |
| `0x200` 起 | 各列四路的累加起始值 |
| `0x400` 起 | 各列四路的计算结果，供读取 |

这张“地址与作用的对应表”来自硬件设计。写C程序时，需要按照它访问。

### 8.2 对照实际Verilog读一小段

打开 pe_axi_wrapper.v，第352行起：

```
else if (awaddr_reg[11:0] == 12'h008) begin
    for (integer b = 0; b < M; b = b + 1)
        if (wstrb_reg[b/8]) sram_ce_reg[b] <= wdata_reg[b];
end
```

先只分三步读：

1. 检查收到地址的末尾是否对应偏移008。
2. 对16个行状态位分别处理，这里M等于16。
3. 把收到的数据位保存到对应行的状态位中。

几个写法逐个解释：

- `awaddr_reg`：暂存收到的写入地址。
- `[11:0]`：只取这个地址最低的12位。
- `12'h008`：Verilog的十六进制写法，数值就是8，前面的12表示占12位。
- `wdata_reg[b]`：收到数据中的第b位。
- `sram_ce_reg[b]`：控制第b行的那个开关。
- `wstrb_reg`：标记这次允许更新哪些字节，先知道它会保护未被选中的字节即可。

对这次0xffff写入，低16位全部是1。相关字节允许更新，因此16个行开关最终全部保存为1。

### 8.3 开关状态怎样接到PE阵列

接口中有实际连接：

```
.sram_ce(sram_ce_reg),
```

按这个格式读：

```
.模块的端口名(接到它上面的信号)
```

这里就是把保存好的行状态，接到PE阵列的sram_ce输入端口。

进入 pe_array.v，第42行，又看到：

```
.sram_ce(sram_ce[row]),
```

它把第row位接给第row行的PE。于是第0位控制第0行，第1位控制第1行，依此类推。同一行的16个PE，共用这一行的开关状态。

## 9. 16列权重怎样送到各个PE

### 9.1 先保存到接口中的寄存器

四次写入完成后，接口模块中的weight_reg暂存着16个权重，全部为1。

每个权重占8位，因此这组数据共 `16×8=128位`。可以想成16个并排的小格子，每个格子装一个权重。

接口中这条连接，把它们送到阵列：

```
.weight_in(weight_reg),
```

### 9.2 阵列每列取出自己的8位

在pe_array.v中：

```
.sram_wdata(weight_in[col*8 +: 8]),
```

这句的作用是：从16个权重中取出第col列的那一个，送给这一列的PE。

`起点 +: 长度` 是Verilog取一段数据的写法：

| col | 起点col×8 | 取出的8位 |
| --- | --- | --- |
| 0 | 0 | 第7～0位 |
| 1 | 8 | 第15～8位 |
| 2 | 16 | 第23～16位 |

所以它们的连接关系是：

```
第0个权重 → 第0列所有PE的权重输入
第1个权重 → 第1列所有PE的权重输入
……
第15个权重 → 第15列所有PE的权重输入
```

第0行、第2列的PE，就会同时收到“第0行的开关状态”和“第2列的权重”。当前两者分别是1和1。

### 9.3 两个模块里相同的变量名要分开看

接口文件和单个PE文件里，都出现了weight_reg。它们是不同模块里的不同寄存器：

| 出现在哪个文件 | 保存的内容 |
| --- | --- |
| `pe_axi_wrapper.v` | 整组16列权重，共128位 |
| `pe_1sram_4mul.v` | 当前这个PE暂存的一个权重，共8位 |

就像两个班都可以有一个叫“小明”的同学，要结合所在的班级区分。读代码时，也要结合当前模块区分名字。

## 10. 单个PE怎样把权重写入、再读出来

### 10.1 sram_wdata是送进来的一个权重

打开 pe_1sram_4mul.v，第8行：

```
input [7:0] sram_wdata,
```

逐个读：input表示这是输入端口；[7:0]表示编号从7到0，共8位；sram_wdata是这组输入的名字。

本测试中，它接收到的8位数据是00000001，也就是权重1。

### 10.2 A负责选位置，D负责送数据

往下找到SRAM的连接，先看两行：

```
.A(sram_addr),
.D(sram_wdata),
```

可以把SRAM想成16个抽屉：

```
sram_addr = 0：选第0号抽屉。
sram_wdata = 1：准备存入数字1。
```

A端口接收抽屉编号，D端口接收要保存的数据。光把这两个信号接好，还需要使能和写入控制，才能真正保存。

### 10.3 哪个时刻真正保存

CPU前面执行的 `wr(0x010, 0xffff)`，会让接口发出一次写入脉冲。某个PE此时的条件是：

| PE侧信号 | 值 | 含义 |
| --- | --- | --- |
| sram_ce | 1 | 这个存储器启用 |
| sram_we | 1 | 当前要写入 |
| sram_addr | 0 | 存到第0号位置 |
| sram_wdata | 1 | 要保存的权重是1 |

条件准备好后，在符合条件的时钟上升沿，存储器把1保存进第0号位置。

因此“准备权重数据”和“触发存储器保存”是两步。它们分别由写权重接口和写0x010控制位置完成。

### 10.4 为什么CEN和GWEN前面有一个波浪号

实际连接里还有：

```
.CEN (~sram_ce),
.GWEN(~sram_we),
```

`~` 表示逐位取反。这里信号只有一位，所以0变1，1变0。

底层SRAM规定：CEN为0时启用；启用后，GWEN为0时写、为1时读。PE侧使用的控制习惯与它相反，所以连接时取反：

```
PE侧sram_ce=1 → 取反后CEN=0 → 存储器启用。
PE侧sram_we=1 → 取反后GWEN=0 → 存储器写入。
PE侧sram_we=0 → 取反后GWEN=1 → 启用时读取。
```

这就是“低有效”的意思：用0表示允许或触发相应操作。

### 10.5 从Q端口读出，再保存到weight_reg

写入脉冲结束后，接口会把sram_we恢复为0。存储器保持启用，在后续时钟上升沿读取所选位置。

输出连接是：

```
.Q(weight),
```

Q是读出数据的端口。第0号位置中保存了1，因此读出后weight这组信号传递的就是1。

随后，前面讲过的时钟代码执行：

```
weight_reg <= weight;
```

在后续时钟沿把读出的权重保存下来，供乘法器使用。需要经过这些时钟阶段，数据才会稳定地走完路径：

```
存储器里的权重 → Q → weight → weight_reg → 乘法器
```

## 11. 跟着输入2，看看一个PE怎样计算

### 11.1 先把四路输入保存下来

实际代码中有：

```
data_reg <= data_in;
```

它位于时钟触发代码块中，表示在规定时刻保存输入。当前每行的输入是0x05040302，稳定后当前PE的data_reg中也保存这四个字节。

第一路从最低8位取数据：

```
data_reg = 05 | 04 | 03 | 02
                          ↑
                data_reg[7:0]取这里，数值2。
```

### 11.2 把2和权重1接到乘法器

实际代码是：

```
Exact_8bit u_mul0 (
    .N1       (data_reg[7:0]),
    .N2       (weight_reg),
    .P_APPROX (product0)
);
```

先看第一行：Exact_8bit是模块类型，u_mul0是放在这里的这个乘法器的名字。可以类比“型号”和“这台设备的编号”。

再看里面的连接：

| 端口 | 连到哪里 | 本次稳定后的值 |
| --- | --- | --- |
| N1 | 第一路输入 | 2 |
| N2 | 当前PE保存的权重 | 1 |
| P_APPROX | product0，接收乘法结果 | 2 |

P_APPROX是源码使用的端口名字。这里先把它理解为这个乘法器的结果出口。

### 11.3 四个乘法器各用哪一份输入

| 乘法器 | 取**data_reg**的哪一段 | 计算 |
| --- | --- | --- |
| u_mul0 | 第7～0位 | 2×1=2 |
| u_mul1 | 第15～8位 | 3×1=3 |
| u_mul2 | 第23～16位 | 4×1=4 |
| u_mul3 | 第31～24位 | 5×1=5 |

四个乘法器共用当前PE的一个权重，各自处理一路输入。这里四个计算可以同时进行。

### 11.4 乘积再加到已有的累加值上

第一路乘法结果经过寄存器保存到product0_reg。传来的第一路累加值保存在psum_reg的最低32位中。

名字中的psum可以理解为“目前加到一半的结果”。

第一路加法器实际代码：

```
CLA32bit u_add0 (
    .A    (psum_reg[31:0]),
    .B    (product0_reg),
    .Cin  (1'b0),
    .Sum  (psum_out[31:0]),
    .Cout ()
);
```

A接已有的累加值，B接本PE的乘法结果，Sum输出新结果。Cin为0表示没有额外的输入进位。Cout留空，表示这里没有连接进位输出。

例如已有结果为6，本PE乘积为2，这个加法器就输出8。

本文件还包含负数乘积的转换处理。当前例子都是正数，先用这些确定的数值理解乘法、暂存和相加的路径。

## 12. 行、列、四路，分别是什么意思

### 12.1 先用一个两行两列的小例子

下面只用来教学，真实工程默认是16行16列。这个小例子先只看一路输入。

假设每行输入都是2，第0列权重是1，第1列权重是2，顶部起始值都是0：

这里特意让同一列各行使用相同权重，便于举例。实际每个PE有自己的权重存储器，也可以选择不同的行分别写入。

| 阵列位置 | 第**0**列：权重**1** | 第**1**列：权重**2** |
| --- | --- | --- |
| 第0行 | 0+2×1=2 | 0+2×2=4 |
| 第1行 | 接到上面的2，再加2，得到4 | 接到上面的4，再加4，得到8 |

这个例子展示两件事：同一列沿行方向向下累加；不同列可以使用不同的权重，得到各自的结果。

阵列里的输入数据从左向右传递，因此各列都能接收到对应行的数据。信号经过寄存器，结果需要时间才能稳定。

### 12.2 回到真实**16**行，为什么第一路输出**32**

真实测试中，每一列权重都设成1。第一路每行输入都是2，因此：

```
顶部起点：0
第0行： 0 + 2×1 = 2
第1行： 2 + 2×1 = 4
第2行： 4 + 2×1 = 6
……
第15行：30 + 2×1 = 32
```

从第0行到第15行，共16行，也就是加了16个2。

### 12.3 另外三路各自相加，不混在一起

四路各有自己的乘法、累加路径：

```
第0路：输入2，每行加2，16行后得到32。
第1路：输入3，每行加3，16行后得到48。
第2路：输入4，每行加4，16行后得到64。
第3路：输入5，每行加5，16行后得到80。
```

四路输入2、3、4、5不会先合成一个数来加。它们得到的是四份独立答案。

### 12.4 为什么共有64个结果

每列底部输出四份答案。一共有16列，所以输出数目是：

```
16列 × 每列4路 = 64个结果。
```

16行决定每路累加多少项；16列决定有多少组输出；四路表示每组同时处理四份输入。

本测试各列权重相同，因此每列的四个结果都是32、48、64、80。

## 13. 结果怎样回到CPU，程序怎样判断成功

### 13.1 最后一行的输出送到接口

pe_array.v把每列最后一行的结果连成整个阵列的输出。接口中有连接：

```
.psum_out(psum_out_wire)
```

于是计算结果出现在接口电路的psum_out_wire信号上。

这里的输出没有另外一套完整结果锁存机制，因此程序需要等待结果稳定，并在读取期间保持相关计算输入稳定。本测试通过前面的配置和等待满足这一用法。

### 13.2 rd通过读取位置选择结果

第一列的四路结果，对应这些32位读取位置：

| 偏移 | 实际地址 | 预期值 |
| --- | --- | --- |
| 0x400 | 0x90000400 | 32 |
| 0x404 | 0x90000404 | 48 |
| 0x408 | 0x90000408 | 64 |
| 0x40c | 0x9000040c | 80 |

CPU执行rd(0x400)，接口根据地址选出包含相应结果的数据，通过AXI返回，CPU取得所需的32位结果。

```
最后一行PE算出32
        ↓
阵列输出 → 接口电路
        ↓ CPU请求读取0x400对应位置
AXI把数据送回CPU
        ↓
rd返回32，程序得到这个数
```

### 13.3 检查语句怎样读

原程序中的第一次检查，展开后相当于：

```
if (rd(0x400) != 32)
    return 1;
```

`if` 表示“如果”；`!=` 表示“不等于”；return表示结束当前函数，并把一个值交还给调用它的地方。

读到32时，32不等于32这个条件不成立，所以继续检查后面的结果。读到30时，30不等于32成立，就返回1，报告失败。

### 13.4 循环怎样把64个结果检查完

实际循环是：

```
for (unsigned col = 0; col < 16; ++col)
    for (unsigned lane = 0; lane < 4; ++lane)
        if (rd(0x400 + 16*col + 4*lane) != 16*(lane+2))
            return 1 + 4*col + lane;
```

col是列号，lane是这列里的路号。每列有四个32位结果，共占16字节，因此列号增加1，起始偏移增加16。

先固定col=0，里面的循环会按下面顺序执行：

| **lane** | 算出的偏移 | 预期值**16×(lane+2)** |
|---|---|---|
| 0 | 0x400 | 32 |
| 1 | 0x404 | 48 |
| 2 | 0x408 | 64 |
| 3 | 0x40c | 80 |

第一列查完，col变成1，lane重新从0开始：从0x410读取32，再依次检查48、64、80。

任何一项不符合预期，程序返回一个非零编号，指出失败位置。所有64项都检查过并正确，才会走到：

```
return 0;
```

这表示main检查成功。启动代码收到main的返回值后，再向仿真环境写入相应的成功或失败标记。

## 14. 在哪些文件里找刚才的内容

| 文件 | 先关注什么 |
|---|---|
| pe_smoke.c | CPU执行的操作顺序，wr、rd和main |
| pe_axi_wrapper.v | 地址如何变成设置，结果如何被读取 |
| pe_array.v | 不同行列的PE怎样连接 |
| pe_1sram_4mul.v | 一个PE中的存储器、寄存器、乘法器和加法器 |
| soc_top.sv | CPU、地址路由和PE的总连接 |
| tb_soc.sv | 仿真用的时钟、复位、外部存储器连接 |

tb_soc.sv属于测试平台，也叫Testbench，简称TB。它在电脑上给待测电路提供运行环境。项目中SRAM的仿真行为模型位于project/tb/sram_reference_model.v。

pe_array.v中的generate循环，是在构建电路时按行列重复放置PE。它最终描述出256个同时存在的PE。

# 15. 我现在能实际做什么：跑测试、找到证据

## 15.1 在终端运行

VS Code顶部选择“终端→新建终端”，依次输入：

```
source /home/zhz/contest/env.sh
```

这行加载主办方准备的工具环境，让终端能找到编译器和仿真工具。

再输入：

```
bash /home/zhz/contest/task1/run_rtl.sh --source /home/zhz/contest/task1/my_design
```

这行启动公开测试脚本，使用你当前的my_design工程副本。脚本自动编译软件、编译仿真模型并运行测试。

这里的--source用于指定工程副本；保留原来的base目录，便于对照。

## 15.2 三组测试分别是什么

| 测试名 | 它在检查什么 |
| --- | --- |
| pe | 单独检查权重存储器，以及不经过CPU的AXI/PE功能 |
| delay0 | 运行整个SoC，外部AXI RAM响应延迟设为0 ns |
| delay1 | 运行整个SoC，外部AXI RAM响应延迟设为1 ns |

两组完整SoC测试中，CPU都会执行前面讲过的C程序并检查64个结果。

全部通过，终端会显示：

```
TASK1_PUBLIC_RTL_ALL_PASS
```

SoC日志里的这句话，直接对应软件检查结果：

```
SOC_SMOKE_PASS: CPU checked all 64 PE output words
```

意思是CPU已经检查了全部64个PE输出，符合预期。

## 15.3 我们已经运行过的结果在哪里

本次会话中，已经使用当前工程副本运行过一次，pe、delay0、delay1三组均通过。

- 本轮结果摘要

- delay0运行日志中的通过标记
- delay1运行日志

重新运行会产生新结果目录，终端用RESULT=给出路径。先找summary.json和运行日志。

摘要中PASS表示本轮检查通过，scope为rtl表示这轮是RTL阶段。baseline为false表示用了自己的工程副本，这个字段本身不表示失败。

这个结果说明公开用例检查通过。后面还需要完成正式综合、时序分析和门级仿真。

### 15.4 波形有什么用，先看什么

波形是“每个信号随时间如何变化”的记录。日志告诉你通过或失败，波形可以帮助你看到过程。

本轮delay0波形可这样打开：

```
source /home/zhz/contest/env.sh
dve -full64 -vpd /home/zhz/contest/task1/work/rtl.0qvuvgs1/delay0/soc.vpd
```

初学时按这个顺序观察：先看clk和rst_n，知道电路何时开始工作；再选一个PE，看它的权重、第一路输入和第一路结果。观察整个阵列结果时，再看底部输出何时稳定为32、48、64、80。

CPU写入、SRAM保存、权重读出、寄存器暂存和最终输出不会全在同一个时刻完成。先找事件之间的先后关系，再研究精确相隔多少周期。

## 16. 读完后，用几个小问题检查自己

问题**1**：**wr(0x008, 0xffff)**中，谁决定访问哪里？

答案：0x008选择PE内部控制位置，实际地址是PE_BASE加这个偏移。

问题**2**：谁决定这次启用哪些行？

答案：写入的0xffff决定。它有16个二进制1，所以全部16行启用。

问题**3**：**wr(0x020, 0x01010101)**一次准备几个权重？

答案：四个。每个权重8位，一次写入32位，所以装四个权重。

问题**4**：为什么下一次写**0x024**？

答案：上次已经覆盖0x020到0x023这四个字节，下一组从0x024开始。

问题**5**：一个**PE**的第一路输入**2**、权重**1**，它自己的乘法结果是多少？

答案：2。它会把这个2加到传入的累加值上。

问题**6**：为什么最后得到**32**？

答案：同一列经过16行，每行加2，起点是0，所以是32。

问题**7**：四路输入**2**、**3**、**4**、**5**是先相加成**14**吗？

答案：它们分别计算，形成四份独立结果32、48、64、80。

问题**8**：**rd(0x400)**的**0x400**是答案吗？

答案：它是读取位置。读到的结果才是答案，本测试应为32。

问题**9**：**Verilog**文件里的多个乘法器，是从上往下依次调用吗？

答案：它们是同时存在的电路，可以并行工作。相关数据由时钟控制的寄存器逐步保存和传递。

问题**10**：看到**RTL**全部通过，比赛任务一就完成了吗？

答案：目前完成了公开RTL测试。任务一后面还要求正式综合、面积和时序报告、SDF以及反标门级仿真，具体见附录B。

## 附录A：完整测试程序，读完前面再看

下面是工程中的完整 pe_smoke.c。只把说明换成中文，程序操作保持一致。它需要使用工程提供的 RISC-V 编译器和运行环境。

```
#include <stdint.h>

#define PE_BASE 0x90000000u
static inline void wr(unsigned off, uint32_t v) {
    *(volatile uint32_t *)(PE_BASE + off) = v;
    __asm__ volatile ("fence iorw, iorw" ::: "memory");
}
static inline uint32_t rd(unsigned off) {
    return *(volatile uint32_t *)(PE_BASE + off);
}
static void settle(void) {
    // 输入保持不变，让 CPU 空转一会儿，等待阵列输出稳定。
    for (volatile unsigned i = 0; i < 128; ++i)
        __asm__ volatile ("nop");
}
int main(void) {
    wr(0x000, 0);                    // 选择权重存储位置0。
    wr(0x008, 0xffff);               // 启用全部16行的权重存储器。
    for (unsigned i = 0; i < 4; ++i)
        wr(0x020 + 4*i, 0x01010101); // 准备16列权重，每个都是1。
    wr(0x010, 0xffff);               // 发出一次写入脉冲，保存权重。
    // 把16列、每列4路的64个累加起始值全部设为0。
    for (unsigned i = 0; i < 64; ++i) wr(0x200 + 4*i, 0);
    for (unsigned row = 0; row < 16; ++row)
        wr(0x100 + 4*row, 0x05040302); // 给每行四路输入2、3、4、5。
    settle();                         // 等待计算稳定。
    for (unsigned col = 0; col < 16; ++col)
        for (unsigned lane = 0; lane < 4; ++lane)
            if (rd(0x400 + 16*col + 4*lane) != 16*(lane+2))
                return 1 + 4*col + lane; // 有结果不对，返回失败编号。
    return 0;                            // 全部64个结果都正确。
}
```

`uint32_t` 表示32位无符号整数。`volatile` 提醒编译器：这里的硬件访问有特殊作用，要按要求执行读写；等待循环的计数也不能被随意消掉。`fence` 用来约束相关读写的先后顺序。

`static inline` 与函数在本文件中的使用、编译时能否展开有关，先照着工程保留即可。`wr`、`rd` 里的指针写法暂时不必逐个符号掌握：前者向“起始地址加偏移”写入32位数据，后者从对应位置读取32位数据并返回。先能看懂调用参数和结果，就可以继续学习。

# 附录**B**：比赛后面要交什么

任务一要回答：这份电路设计能否转换成具体电路，并在考虑延迟后继续正确工作。 主办方已给出起始设计，本题不要求制造芯片或布局布线。

第一步叫“RTL前仿”：电脑按现有电路代码模拟运行，让CPU执行测试程序。VCS是做这件事的工具，DVE用来看信号波形。我们已经运行了现有三组公开测试，结果通过；这证明相应用例正确，后面还有工作。

第二步写“时序约束”，保存成SDC文件。可以把它理解为给电路定工作要求，例如时钟多快、外部数据什么时候到。工具需要这些条件，才能选择合适的电路实现。正式要求以主办方说明为准。

第三步“逻辑综合”：Design Compiler（常简称DC）读取电路代码和标准单元库，挑选并连接具体电路单元。标准单元库像零件目录，记录每种单元的功能、面积和延迟。输出的门级网表说明用了哪些单元、怎样连接；面积报告说明使用了多少硬件资源。

工程中的frontend-synth-check是通用综合检查。要满足比赛任务，还需要使用规定的标准单元库完成映射，得到正式面积、时序和门仿结果。

第四步检查时序。寄存器按时钟接收数据，前面的计算需要及时完成。“静态时序分析”，简称STA，就是检查这些时间要求。PrimeTime（简称PT）是相关工具。要保存时序报告，查看关键路径和不满足要求的地方，再处理影响正确工作的违规。

用一个时间例子理解：时钟每10 ns来一次，对应100 MHz。假设某条路径扣除其他要求后，留给数据9 ns；数据实际需要7 ns，就剩2 ns裕量。以建立时间检查为例，如果实际需要11 ns、只允许9 ns，裕量就是-2 ns，需要处理。这里的数值用于教学，实际约束要按正式规则确定。

第五步生成SDF延迟文件，并运行门级仿真。把延迟加载进去叫“SDF反标”。这时使用综合网表、单元和SRAM模型、SDF及测试平台，再运行测试，检查结果与前仿是否一致。还要确认延迟正确加载。现有`delay0`和`delay1`改变的是外部存储器响应时间，不能替代这一步。

SRAM在综合时使用黑盒接口和独立库信息，在仿真时需要相应的读写模型。综合黑盒只描述边界，不能直接当作能工作的仿真存储器。

最终需要整理网表、SDC、面积和时序报告、SDF，以及前仿和门仿的日志、波形、检查结果与运行方法。本次已经完成RTL用例验证，正式综合、时序分析和SDF门仿仍需继续完成。

任务二要让整张神经网络在**RISC-V**与**PE**系统上完成推理。 包括加载权重、搬运输入、调度计算、处理各层结果，最后与参考答案核对。Python参考程序提供对照结果，目标系统上的这些工作仍要实现。

模型INT8权重约2.73 MiB，基础固件空间只有256 KiB，无法照原来大小一次装下全部权重。因此还要安排存储位置，以及分块加载和计算。

正确后再优化。主要指标ADP是“SoC面积乘完整推理时间”；这个时间包含数据搬运、计算、软件调度和结果输出。加硬件可能更快，也会增加面积，要一起比较。新增硬件要纳入综合。模型精度、时钟和面积统计等验收要求，仍以主办方正式规则为准。

举个比较用的例子：方案一面积100、时间10，ADP为1000；方案二面积150、时间8，ADP为1200。方案二虽然更快，但方案一的面积与时间乘积更小。这些数值仅帮助理解指标，不是当前工程的实际成绩。